

MEMORIA DE PROYECTO DE GRADO

PROGRAMA PARA RESOLVER

LABERINTOS

ENTORNOS DEL DESARROLLO
CURSO ACADÉMICO 2025/26: 1º DAM



Alumno: Daniel García Rodríguez
Profesor: David Munuera Sánchez

22 de Mayo de 2026

ÍNDICE

1. Introducción	2
2. Etapas del desarrollo	3
2.1 Primera entrega	3
2.2 Segunda entrega	5
2.3 Tercera entrega	6
2.4 Cuarta entrega	7
3. Definición del funcionamiento básico	8
3.1 Ejecución completa del programa	8
3.2 Situaciones excepcionales	16
4. Definición en UML del programa	19
4.1 Diagrama casos de uso	19
4.1.1 Fichas casos de uso	21
4.1.2 Diagrama de clases	27
5. Definición individual de cada clase del programa	28
5.1 Clase Main	28
5.2 Clase Session	29
5.3 Clase User	30
5.4 Clase Maze	31
5.5 Clase Input	34
5.6 Clase DAO	36
5.7 Clase Utils	37
5.8 Clase Coordinate	39
5.9 Clase Config	39
6. Desarrollo de la implementación de cada funcionalidad individual del programa	40
6.1 Funcionalidades en Estado No Autenticado	40
6.1.1 Iniciar Sesión (Login)	40
6.1.2 Registro (Signup)	41
6.1.3 Cerrar programa	42
6.2 Funcionalidades en Estado Autenticado	43
6.2.1 Cargar Laberinto	43
6.2.2 Ver Laberinto	44
6.2.3 Establecer Coordenadas de Entrada y Salida	45
6.2.4 Seleccionar camino	46
6.2.5 Ver usuario actual	47
6.2.6 Cerrar Sesión (Logout)	47
7º Desarrollo de la implementación del algoritmo del primer camino.	48
8. Desarrollo de la implementación del algoritmo del camino más corto	49
9º Conclusiones	51
10º Bibliografía	51

1. Introducción

Esta memoria muestra el proceso de construcción de un programa diseñado por un alumno de 1º de Desarrollo de Aplicaciones Multiplataforma (DAM), donde se plantean una serie de pautas a seguir en cada una de las fechas que se han ido marcando durante el transcurso de este curso.

En esta memoria se explicará el funcionamiento de cada versión del programa, desde la primera versión v0.1.0 (cuya entrega se realizó el 11 de enero del 2026), hasta la versión v1.1.0 (entregada el 12 de abril de 2026). En cada una de las entregas, la dificultad de los objetivos marcados por el profesor crece, ya que cada entrega requiere de las lecciones vistas en clase hasta el momento, por lo que se verá un crecimiento exponencial desde la entrega v0.1.0 a nivel de funcionalidad y profesionalidad en el código, hasta la entrega v1.1.0, donde se muestra un programa modular, que usa una metodología más eficiente que las entregas iniciales.

La idea principal de este proyecto se basa en la creación de un programa capaz de encontrar el camino adecuado para salir de cualquier laberinto que se introduzca en él, además de mostrar información acerca de los laberintos que haya sido capaz de resolver. Este programa también tiene que tratar con cuentas de usuarios, ya que cada persona que tenga intención de usar "MazeSolver", deberá hacer un registro, donde tendrá que introducir un nombre de usuario, una contraseña e información personal. En cuanto al registro e inicio de sesión, se han ido implementando una serie de restricciones, controles de formato, tratamientos de excepciones y capas de seguridad dentro del programa, de manera que todo el flujo de información que se realiza dentro del programa, tenga un orden que permita asegurar un programa consistente.

Por otro lado, para que el programa sea capaz de resolver laberintos, es necesaria la implementación de un algoritmo capaz de crear una sucesión de pasos, con una lógica interna que permita formar una vía de escape. En este proyecto se ha optado por usar un algoritmo de búsqueda en profundidad (DFS), para formar el primer camino que encuentre el algoritmo (siendo este un camino no tan óptimo en la mayoría de los casos) y un algoritmo de búsqueda en anchura (BFS), para formar el camino más corto posible de cualquier laberinto introducido (este camino sí es el más óptimo, formado por la menor cantidad de pasos). En los apartados dedicados a la explicación del funcionamiento del programa, entraré en detalles de cómo es la lógica de estos algoritmos.

2. Etapas del desarrollo

2.1 Primera entrega

La primera entrega del proyecto tuvo lugar el 11 de enero del 2026, en la cual tuvimos que seguir una serie de indicaciones del profesor para la creación de un menú de inicio de sesión y registro de usuarios, así como la creación de los "cimientos" del programa, para la preparación de futuras entregas mediante la implementación de varias clases dedicadas (**Config**, **User**, **Session**).

En primer lugar, desde la clase **Main** del proyecto, se gestionan los dos menús que formarán el programa de la primera entrega, y hay que diferenciarlos en función del propósito y funcionamiento de cada uno. El primer menú que el usuario visualizará al iniciar el programa será el menú anterior al inicio de sesión (unLoggedMenu), el cual ofrece la posibilidad al usuario del programa de crear una cuenta nueva a través del registro, o iniciar sesión con una cuenta existente dentro del fichero (users.txt) que actúa como base de datos.

El menú ofrece la posibilidad a usuarios que ya han sido registrados anteriormente de disfrutar de la funcionalidad del programa, y también permite crear nuevas cuentas a nuevos usuarios. Sin embargo, esta versión del menú inicial del programa carece de validaciones de formato, encriptación de contraseña, y el almacenamiento de los datos de los usuarios del programa se gestiona desde un fichero de texto plano (users.txt).

Para tratar con las sesiones de los usuarios, es necesario implementar la clase **Session** para realizar la comprobación de las credenciales de los usuarios, de manera que se acceda al fichero de los usuarios del programa y se extraiga la información necesaria para certificar la validez de los parámetros introducidos por el usuario. En caso de que los datos introducidos en la opción de login sean válidos, el programa dará paso al siguiente menú del programa.

En cuanto al registro de nuevas cuentas, el usuario que tiene la intención de usar el programa deberá introducir los siguientes datos: nombre de usuario, contraseña, nombre completo, NIF, email, dirección y fecha de nacimiento. Dichos datos se almacenarán dentro del fichero de texto plano, todos escritos en una misma línea y separados por ';'. Esta práctica no es realista, ya que se solicita al usuario una serie de datos que carecen de métodos que aseguren su fiabilidad, dado que las expresiones regulares se implementarán en futuras entregas. A esto se añade la inexistencia de una base de datos real, que garantizaría un acceso sencillo a los datos, una organización esencial para el funcionamiento interno del programa y las capas de seguridad necesarias en programas que tratan con información sensible de sus usuarios, como por ejemplo la contraseña.

Pasando al menú de sesión iniciada, se mostrarán las siguientes opciones:

- **Cargar laberinto:** El objetivo de esta opción será el de cargar uno de los distintos archivos que contendrán laberintos a resolver. Sin embargo, en esta primera entrega simplemente mostrará el mensaje "Próximamente", indicando que su funcionalidad será desarrollada en futuras entregas.

- **Ver laberinto actual:** El objetivo de esta opción será procesar el archivo que se habrá cargado anteriormente en la opción de cargar laberinto, para presentar una imagen del laberinto al usuario del programa. En esta primera entrega simplemente mostrará el mensaje “Próximamente”, indicando que su funcionalidad será desarrollada en futuras entregas, al igual que ocurre en la anterior opción.
- **Establecer casillas de entrada y salida:** Esta funcionalidad del programa permitirá que el usuario del programa establezca las casillas de entrada y salida mediante la introducción de las coordenadas X e Y, pero aún no se ha llevado a cabo su desarrollo dentro de esta entrega (Próximamente).
- **Buscar caminos:** Esta opción en esta primera entrega mostrará el mensaje “Próximamente”. Adelantando un poco qué papel tendrá dentro del programa, “buscar caminos” será la encargada de brindar dos posibilidades para llevar a cabo la resolución de cualquier laberinto seleccionado:

1º Resolver laberinto implementando el algoritmo DFS, el cual dará como resultado el primer camino que encuentre, que cumpla la condición de llegar desde la casilla de inicio hasta la salida. El camino generado no es el más óptimo posible, ya que depende de qué tipo de laberinto se cargue y de la posición de las casillas establecidas (Opción que se desarrolla posteriormente).

2º Resolver laberinto implementando el algoritmo BFS, el cual dará como resultado la trazada del camino más corto posible (Opción que se desarrolla posteriormente).

- **Ver usuario actual:** Se ofrece la posibilidad de visualizar los datos del usuario que se encuentra dentro del programa con la sesión iniciada, donde se extraerá la información de esa persona del fichero de datos, y se mostrará de manera ordenada, estructurada y fácilmente legible.
- **Cerrar sesión:** Por último, el menú de sesión iniciada, ofrece la posibilidad de regresar al menú principal a través de esta funcionalidad. Cuando se elige esta opción, el programa convierte la sesión iniciada en sesión no iniciada, regresando al menú donde el usuario deberá volver a iniciar sesión, registrar una cuenta nueva o finalizar el programa con la opción de salir del programa.
- **Salir:** Permite cerrar el programa desde el menú de sesión iniciada (requiere confirmación).

Volviendo a las clases dedicadas, la clase **Session** es la que tiene mayor importancia dentro de esta entrega, pero también hay que tener en cuenta la función que cumplen la clase **User** y la clase **Config**. La clase **User** será la encargada de crear objetos de tipo usuario, permitiendo definir y organizar los atributos de cada uno para simular el funcionamiento de un programa real y bien estructurado. La clase **Config** almacena atributos estáticos que corresponden a la configuración del programa, como pueden ser: versión del programa, mensaje de bienvenida, mensaje al finalizar el programa, menú "UnLogged" y menú "LoggedMenu".

2.2 Segunda entrega

La segunda entrega del proyecto tuvo lugar el 4 de febrero del 2026, en la cual se aumentó la funcionalidad del programa, ya que la versión v0.1.0 de la entrega anterior estaba muy alejada de ser una versión funcional para un usuario normal.

En esta versión v0.2.0 se mantiene el uso de un fichero de texto para el almacenamiento de datos de los usuarios del sistema, pero se da un paso bastante importante en cuanto a usabilidad del programa, ya que se desarrollan tres de las cuatro funcionalidades que le dan sentido al nombre del programa y a su razón de ser. Este avance se ve reflejado en la clase **Maze**, la cual se estructura con multitud de métodos que permiten la implementación de las tres primeras opciones dentro del menú de usuarios con sesión iniciada.

Como es lógico, el código desarrollado en la primera entrega se reutiliza en esta nueva entrega, donde se deben adaptar ciertos aspectos, como el cambio de versión v0.1.0 en la clase **Config** o la creación de una carpeta donde se almacenarán los laberintos a resolver, entre otros.

La clase **Main** mantiene su esencia, siendo la encargada de mostrar los dos menús (unLogged y Logged) y realizar las llamadas pertinentes a los distintos métodos de las clases dedicadas.

La clase **Session** no presenta cambios sustanciales, ya que las modificaciones llevadas a cabo en esta entrega no tienen relación directa con el tratamiento de datos del usuario. Lo mismo ocurre con la clase **User**, cuyas modificaciones y agregaciones de código no requieren cambios en esta clase.

La clase **Maze** es la que abarca prácticamente toda la carga de trabajo de esta entrega, ya que supone la creación de una clase desde cero, con métodos orientados a cargar los distintos laberintos (procesando los ficheros de texto proporcionados por el profesor), generar una vista del mapa seleccionado por el usuario y establecer las casillas de entrada y salida indicadas por el mismo.

Una vez desarrolladas las pautas de esta entrega, las opciones dentro del menú de sesión iniciada quedarían de la siguiente manera:

- **Cargar laberinto:** El programa, en este punto, crea un menú donde muestra al usuario los distintos mapas disponibles para elegir. Este menú permite al usuario escribir el número del archivo que más le interese, y cuando esto ocurra, el programa relaciona la opción elegida con el archivo de texto que le corresponda. Internamente se prepara un “mapa” que permite que la opción **ver laberinto actual**, pueda generar la vista mencionada anteriormente, del laberinto elegido.
- **Ver laberinto actual:** Aquí es donde el programa crea, gracias a la preparación de **cargar laberinto**, una imagen exacta del laberinto que se ha seleccionado entre los archivos de texto de la carpeta de laberintos. Esta imagen sustituye los caracteres del archivo de texto original (#, ‘ ’) por emoticonos, que mejoran el diseño original, y permiten que la resolución de los laberintos sea más agradable a la vista del usuario. Asimismo, se crea una numeración en las filas y columnas del laberinto, que será clave en el funcionamiento del

programa, ya que el usuario también debe introducir las coordenadas de entrada y salida del laberinto para su posterior solución.

- **Establecer casillas de inicio y fin:** El último apartado de esta entrega, requiere del desarrollo de una opción que permita al usuario seleccionar una entrada y una salida, para cualquier laberinto cargado. Esta funcionalidad pide dos coordenadas al usuario:

1º Coordenadas de inicio: En primer lugar la coordenada del inicio, donde el usuario decidirá dónde será el lugar de partida del algoritmo resolutor de laberintos, por lo que se tendrá que apoyar de la opción de mostrar laberinto, para elegir cuáles son las coordenadas elegidas.

2º Coordenada de final/salida: En segundo lugar, se solicitan las coordenadas que formarán la salida del laberinto.

Una vez determinadas las casillas, se establecen en el mapa interno del laberinto, de manera que si el usuario vuelve a pulsar la opción de mostrar el laberinto, se generará de nuevo la visualización del mapa con las casillas ya establecidas. En caso de que el usuario introduzca coordenadas no válidas, el programa responderá mediante el tratamiento de excepciones implementado en esta entrega, y el mapa interno no se modificará hasta que se confirme que todos los datos son correctos.

2.3 Tercera entrega

La tercera entrega del proyecto tuvo lugar el 25 de febrero del 2026, donde se formaliza la versión v1.0.0, de la que ya se puede hablar como un programa con la funcionalidad mínima para ser considerado completo. Para lograrlo, se han implementado los algoritmos mencionados previamente, capaces de resolver los laberintos registrados dentro del sistema.

En base a la estructura del programa que se ha ido formando con las diferentes entregas, se ha creado una nueva clase llamada **Coordinate**, cuya función es instanciar objetos de coordenadas para almacenar todos los puntos referentes al laberinto. A simple vista puede parecer una clase secundaria dado su reducido volumen de código, pero constituye el pilar fundamental de la resolución de los laberintos, ya que los algoritmos operan con objetos de tipo coordenada en todo momento.

Siguiendo la misma línea, se han desarrollado métodos dedicados para permitir que el programa sea capaz de dar respuesta a los objetivos que se le marquen durante su funcionamiento. Para entenderlo mejor, nos centraremos en las opciones añadidas dentro del menú de sesión iniciada en el apartado "Buscar caminos", donde el usuario podrá elegir entre obtener el primer camino que ofrezca un resultado o el camino más eficiente:

- 1º Primer camino: Si se elige esta opción, el programa aplica los métodos dedicados al algoritmo de búsqueda en profundidad o DFS (Depth-First Search). El comportamiento de este algoritmo consiste en probar una ruta siguiendo un orden de preferencia de movimientos establecido para explorar secuencialmente cada paso (primero trata de moverse a la derecha, luego a la izquierda si no ha podido avanzar, etc.), y continúa por esa ruta hasta que no existen más posibilidades de avanzar por esa ramificación del laberinto. Cuando encuentra un camino que le permite alcanzar la meta, el algoritmo

finaliza su procesamiento. Este algoritmo presenta la ventaja de requerir menos memoria que el empleado para el camino más corto; sin embargo, el resultado no es el más eficiente en cuanto a distancia recorrida en la mayoría de los casos.

- **2º Camino más corto:** Si se elige esta opción, el programa aplica los métodos dedicados al algoritmo de búsqueda en anchura o BFS (Breadth-First Search). El comportamiento de este algoritmo consiste en recorrer el laberinto por niveles de distancia, de manera que no se centra en un único camino, sino que abarca varias posibilidades simultáneamente. El primer candidato que alcance la casilla de salida será matemáticamente el camino que menos pasos requiere para resolver el laberinto. Este algoritmo tiene la ventaja de garantizar con total precisión el camino más corto; sin embargo, se recomienda su uso cuando la búsqueda se realiza en grafos de poca profundidad, es decir, cuando la entrada y la salida se encuentran relativamente cerca, ya que este algoritmo almacena en memoria todas las posibles coordenadas hasta encontrar la salida. En laberintos de grandes dimensiones, la gestión de memoria podría convertirse en un problema. En este programa, no obstante, no se presenta dicho inconveniente, ya que no sería razonable introducir un laberinto de tales dimensiones.

En cuanto al resto de las clases, no se han realizado cambios significativos, manteniéndose la gestión de usuarios mediante ficheros de texto plano en esta fase. Sin embargo, este es el punto de inflexión del proyecto: una vez consolidada la lógica central y los algoritmos de resolución, se dará el salto en la última entrega hacia un sistema de gestión de datos respaldado por una base de datos MySQL. Este cambio permitirá una persistencia de datos profesional, dotando al programa de una robustez y escalabilidad propias de una aplicación real.

2.4 Cuarta entrega

La cuarta entrega del proyecto tuvo lugar el 12 de abril del 2026, donde se dio un salto en cuanto a profesionalidad y robustez técnica con la sustitución del sistema de almacenamiento basado en ficheros de texto plano por una base de datos relacional MySQL. Este cambio en la gestión de los datos supone un avance significativo hacia lo que sería una aplicación real, ya que permite una gestión de la información mucho más segura y eficiente, superando las limitaciones que presentaban los archivos de texto plano.

Para llevar a cabo la implementación de la base de datos, lo primero fue la reestructuración de todos los métodos encargados del tratamiento de datos, ya que estos mantenían vínculos directos con el archivo de texto users.txt, el cual ha desaparecido por completo en esta entrega. El profesor estableció en los requisitos de la entrega, de manera clara y precisa, las características que debía tener la base de datos a crear.

La base de datos "Maze" contendrá una tabla de usuarios llamada user, con los siguientes campos:

ID	Actúa como la Primary key de los usuarios de la base de datos.
----	---

Username	Nombre de usuario que no se podrá repetir dentro de la base de datos.
Password	Este campo es la contraseña de los usuarios registrados.
Name	Este campo formará el nombre completo de los usuarios del sistema.
Nif	NIF/NIE de los usuarios, no se podrá repetir dentro de la base de datos.
Email	Correo electrónico de los usuarios, no se podrá repetir en la base de datos.
address	Dirección completa de los usuarios, este campo puede no tener valor (null).
birthdate	Fecha de nacimiento de los usuarios, este campo puede no tener valor (null).
role	Indica el rol dentro del programa de los usuarios (user por defecto).

Una vez creada la base de datos y asignados los campos necesarios para el almacenamiento de todos los atributos del usuario, se procedió a crear una clase dedicada capaz de gestionar la conexión del programa con la base de datos MySQL. Esta clase recibe el nombre de **DAO** (Data Access Object) y permite que el programa sea fácil de mantener gracias a la reutilización de código, además de ofrecer una mejor escalabilidad futura, al centralizar en ella toda la gestión de conexiones con la base de datos, separando esta responsabilidad del programa principal y actuando como intermediario entre las acciones que se realizan en el programa y las consultas necesarias en la base de datos. Esto también añade una capa de protección adicional en el tratamiento de datos, facilitando la protección frente a inyecciones de código SQL no deseadas.

La implementación de la clase **DAO** implica la modificación de varias clases dentro del programa, dado que se han creado métodos para consultar información específica dentro de la base de datos.

3. Definición del funcionamiento básico

3.1 Ejecución completa del programa

En este apartado de la memoria, se va a documentar el flujo de ejecución completo del programa. Este flujo de ejecución se corresponde con la última entrega reflejada en esta memoria (v1.1.0) y se divide en estos apartados:

1º Inicio del programa
2º Registro de usuarios
3º Inicio de sesión
4º Interacción con laberintos
5º Buscar laberinto
6º Ver información usuario actual
6º Cierre de sesión y de programa

1º Inicio del programa (Mensaje de bienvenida y menú UnLogged)

Cuando un usuario inicie el programa, la primera interacción que va a tener con el programa es la siguiente:

```
!Bienvenido; Este es el programa diseñado por Daniel García 1ºDAM
Version de programa: v1.1.0
```

```
1- Iniciar sesion
2- Registrarse
0- Salir
```

Opcion:

El usuario tendrá la posibilidad de iniciar sesión (si dispone de una cuenta creada), o podrá comenzar el proceso de registro de usuarios. En el hipotético caso de que el usuario nunca haya usado el programa, escribirá en el programa la opción 2, y se cargará el proceso de registro mencionado.

2º Menú de registro de usuarios

Una vez que se haya seleccionado la opción 2 de registro, el programa solicitará al usuario, de forma progresiva, los datos necesarios para formalizar el registro dentro de la base de datos. Este proceso de registro está respaldado de restricciones de formato, para asegurar que la acción de insertar nuevos usuarios en base de datos sea limpia y no genere excepciones ni errores en este procedimiento.

```
Opcion: 2
Introduce nombre de usuario: Prueba1
Contraseña: UsuarioPrueba1
Confirma la contraseña: UsuarioPrueba1
Introduce tu nombre completo: Usuario de prueba
Introduce tu DNI/NIF: 99999999V
Introduce tu correo electrónico: usuarioprueba@gmail.com
Introduce tu dirección: Calle falsa
Introduce tu fecha de nacimiento: 01/01/2000
```

En caso de que todos los datos introducidos sean válidos (como se observa en el ejemplo de la imagen), el programa hará un intento de inserción dentro de base de datos y, en caso de éxito, mostrará un mensaje mostrando que el objetivo que tenía el usuario de registrar una cuenta nueva se ha cumplido.

```
-----Usuario creado con éxito-----
```

3º Menú de inicio de sesión

En el momento en el que el usuario ya dispone de una cuenta creada, ya tiene la posibilidad de entrar a la funcionalidad completa del MazeSolver. Para ello, tendrá que seleccionar la opción 1 dentro del menú Unlogged para comenzar con este proceso.

Opcion: 1

Nombre de usuario: Prueba1

Contraseña: UsuarioPrueba1

Pulse 'enter' para continuar

Una vez que se introduce un usuario y contraseña se hace la comprobación dentro de la base de datos, y como es un usuario que se ha registrado previamente dentro del programa, se dará paso al siguiente menú (LoggedMenu):

Opcion: 1

Nombre de usuario: Prueba1

Contraseña: UsuarioPrueba1

Pulse 'enter' para continuar

- 1- Cargar laberinto
- 2- Ver laberinto
- 3- Establecer casillas de entrada y salida
- 4- Buscar caminos
- 5- Ver usuario actual
- 6- Cerrar sesión
- 0- Salir

Opción:

Cuando tengamos la vista de este menú significa que el estado de la sesión dentro del programa pasa de ser logged = false a logged = true, dándonos "permiso" para acceder a las funcionalidades del programa. Vamos a ir paso a paso en la ejecución del programa, por lo que voy a simular que seleccionamos la opción 1 (Cargar laberinto).

4º Interacción del usuario con los laberintos del sistema.

En este apartado vamos a englobar toda la funcionalidad referente a la elección, visualización, colocación de casillas de entrada y salida, de los laberintos que se almacenan dentro del programa. En primer lugar seleccionamos la opción 1 (Cargar laberinto).

Opción: 1

--- Seleccione un laberinto ---

- 1º Dificil.txt
- 2º Medio.txt
- 3º Facil.txt
- 4º Distinto.txt

Selecciona un laberinto: 1

El usuario deberá elegir uno de los distintos laberintos posibles poniendo el número que le corresponde a dicho laberinto, simulamos la elección del laberinto 1 (Dificil.txt) y este sería el mensaje que aparecerá cuándo se elige una opción válida:

Mapa cargado correctamente!!

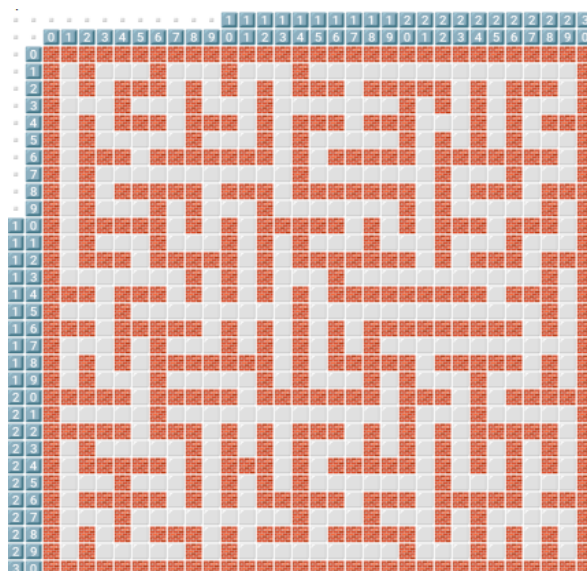
Pulse 'enter' para continuar

- 1- Cargar laberinto
- 2- Ver laberinto
- 3- Establecer casillas de entrada y salida
- 4- Buscar caminos
- 5- Ver usuario actual
- 6- Cerrar sesión
- 0- Salir



Opción: 2

El siguiente paso será la comprobación de que el programa es capaz de mostrar el laberinto seleccionado, para ello elegimos la opción 2 del menú Logged (Ver laberinto).



Cuándo el usuario ha comprobado que el laberinto se ha cargado completamente y tiene una vista del mismo, elegirá la opción 3 (Establecer casillas de entrada y salida), donde el programa solicitará individualmente las coordenadas X e Y de la entrada y salida.

Opción: 3

Para establecer la entrada indica las coordenadas

Valor de I: 1

Valor de J: 1

Para establecer la salida indica las coordenadas

Valor de I: 29

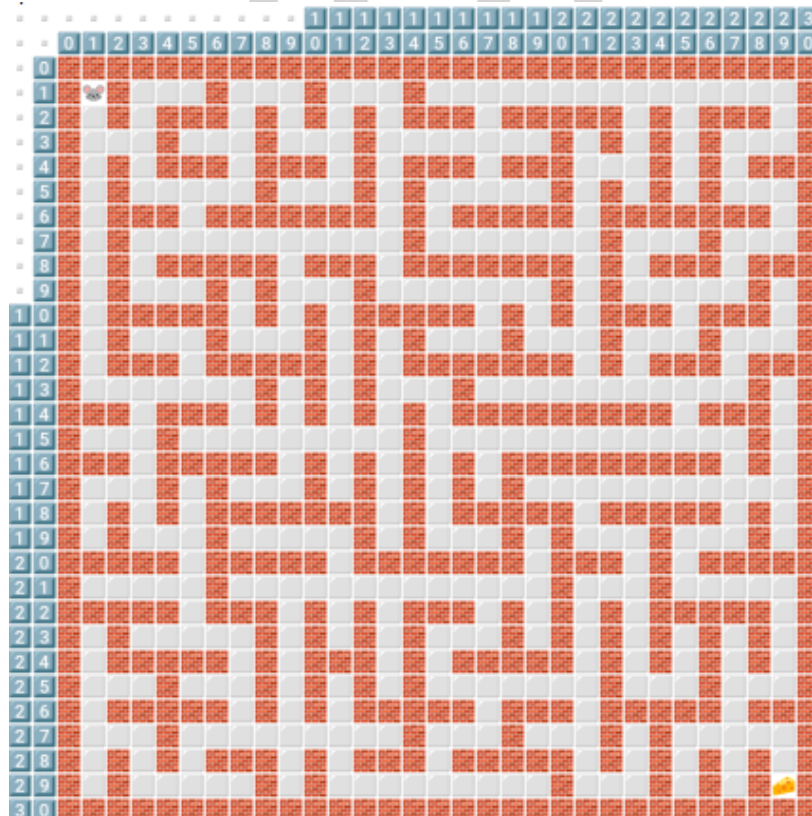
Valor de J: 29

Si las coordenadas introducidas son válidas y no corresponden a la posición de un muro dentro del mapa, el sistema mostrará un mensaje de éxito.

Entrada y salida integrada correctamente!!!

Pulse 'enter' para continuar

Cuando hayamos establecido las casillas de entrada y salida, podemos volver a seleccionar la opción 2 (Ver laberinto), para comprobar que las casillas realmente aparecen reflejadas en el mapa que el programa procesa para la opción de buscar camino.



5º Búsqueda de un camino

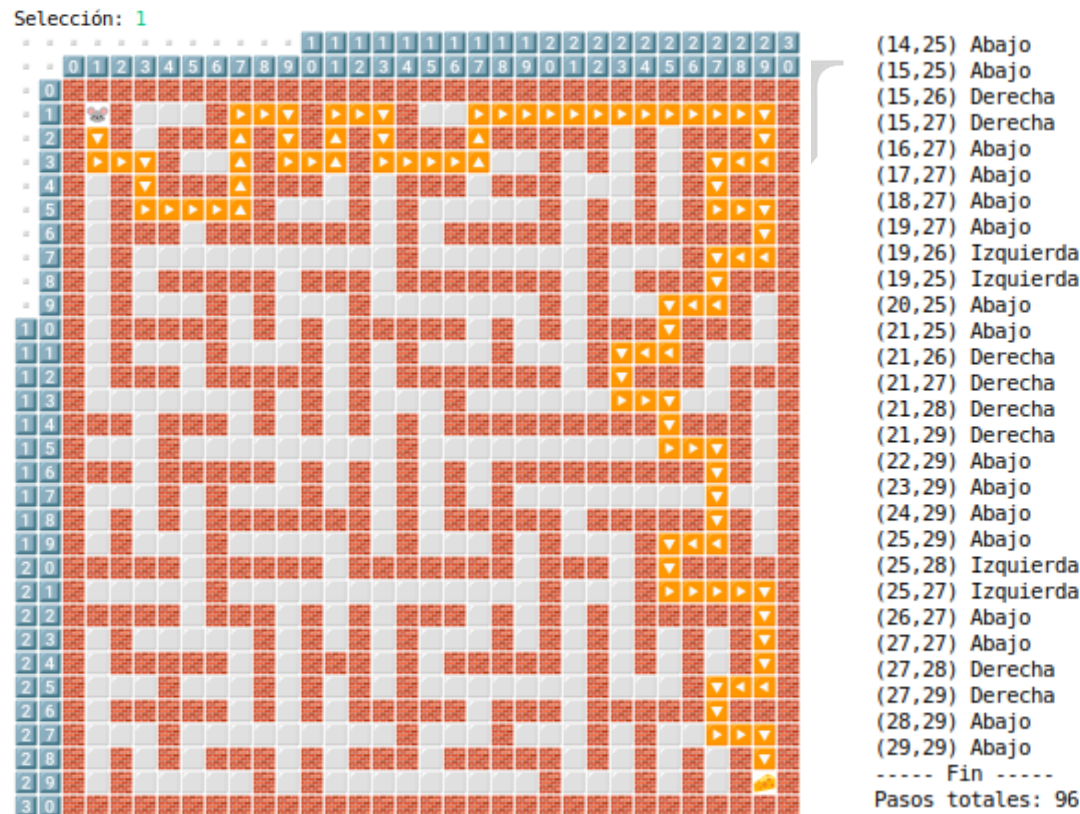
En este punto tenemos el laberinto cargado en el sistema y el usuario ha creado a su elección una casilla de entrada y otra de salida, solo queda elegir que tipo de algoritmo va a entrar en funcionamiento. Para hacer esta elección, debemos entrar en la opción 4 (Buscar camino).

```
Opción: 4
--- SELECCIONA UNA OPCIÓN ---

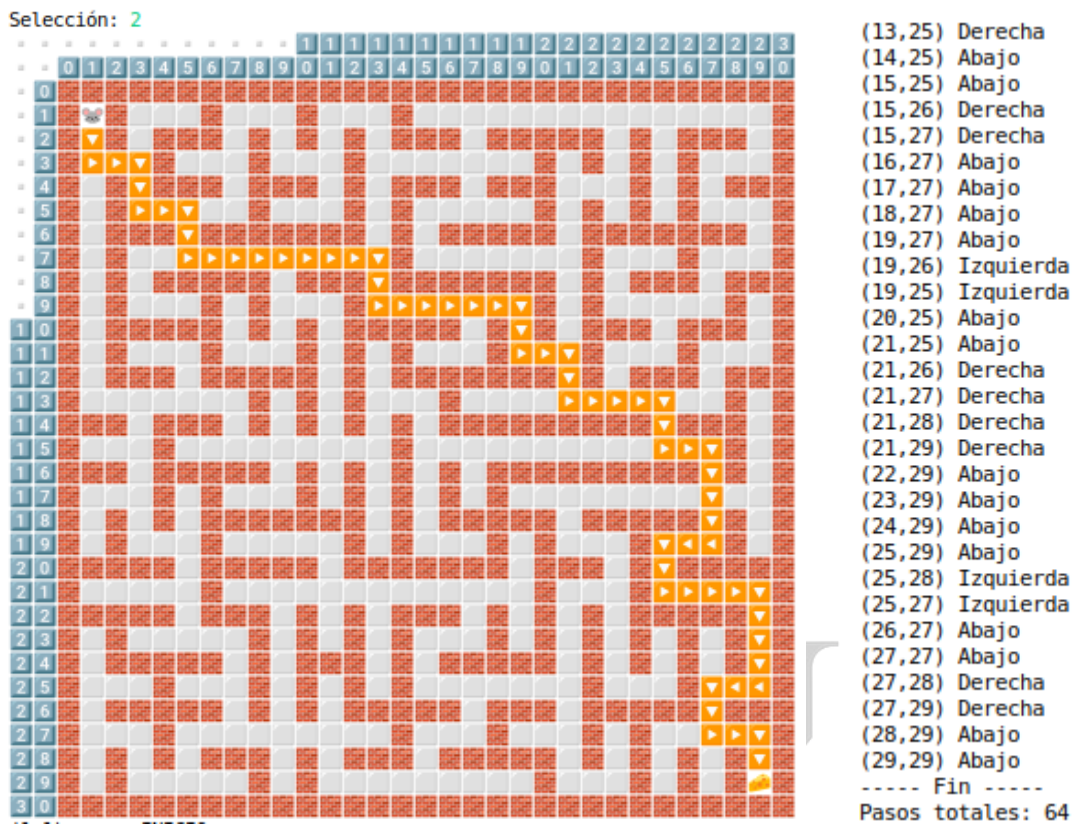
1º Encontrar cualquier camino
2º Camino más corto
0 Cancelar
```

Selección:

Cuando el usuario seleccione la opción 4 se desplegará el menú mostrado en la imagen superior. La opción 1 dentro de este menú, aplicaría un algoritmo DFS (Búsqueda profunda), mientras que la opción 2 aplica un algoritmo BFS (Búsqueda en anchura). En primer lugar, simulamos la opción 1 (Encontrar cualquier camino).



Este es el resultado de la opción de obtener cualquier camino (Primero que el algoritmo obtiene), vamos a probar la otra opción posible, para comparar los resultados de los dos algoritmos. Para ello volvemos a entrar en buscar camino y seleccionamos esta vez la opción 2.



Y este sería el resultado del algoritmo BFS de la opción 2 (Camino más corto), donde el resultado es sustancialmente menor, demostrando que realmente el primer algoritmo usado está lejos de ser el algoritmo más óptimo en cuanto a distancia.

6º Ver información del usuario actual.

También está la posibilidad de que el usuario quiera ver la información sobre su cuenta, por si no se acuerda del correo, dirección, o cualquier otro campo de información con la que se registró, por lo que en el menú de sesión iniciada existe la opción de ver usuario actual. Vamos a entrar en esta opción 5. Dentro de esta opción se ha implementado también la posibilidad de ver la edad en años de los usuarios de las cuentas, a través de un cálculo que toma de referencia la fecha de nacimiento y la compara con la fecha actual, dando como resultado lo que se muestra en la imagen en la parte de abajo.

Opción: 5

---USUARIO ACTUAL---

Usuario: Prueba1

Id de usuario: 20

Nombre completo: Usuario de prueba

Nif/DNI: 99999999V

Correo electrónico: usuarioprueba@gmail.com

Dirección: Calle falsa

Fecha de nacimiento: 01/01/2000 --> Edad: 26 años

Rol de la cuenta: user

7º Cierre de sesión y de programa

Cuando el usuario ya ha usado toda la funcionalidad del programa, muy probablemente querrá que su cuenta se cierre, por lo que tendrá que seleccionar la opción de cierre de sesión dentro del menú Logged (opción 6).

- 1- Cargar laberinto
- 2- Ver laberinto
- 3- Establecer casillas de entrada y salida
- 4- Buscar caminos
- 5- Ver usuario actual
- 6- Cerrar sesión
- 0- Salir

Opción: 6

---Sesión cerrada---

Cuando se selecciona la opción de cerrar sesión, internamente el programa convierte el booleano **logged = true** en **logged = false**, y esto genera en el programa que el objeto usuario que se estaba usando se convierta en null, mostrando el mensaje que se ve en la imagen, para indicar al usuario que su deseo se ha cumplido con éxito. Ahora sólo faltaría cerrar por completo el programa, y se podría hacer seleccionando la opción 0 en el menú Logged (en esta posibilidad, el programa pide confirmación para realizar la acción).

Opción: 0

¿Seguro que quieres salir del programa? Pulsa 0 para confirmar o 1 para cancelar: 0

¡Hasta la próxima!
FIN DEL PROGRAMA

Otra posibilidad para cerrar el programa por completo sería elegir la opción 6 (Cerrar sesión) en el LoggedMenu, y posteriormente elegir la opción 0 en el UnloggedMenu.

Opción: 6

---Sesión cerrada---

1- Iniciar sesion
2- Registrarse
0- Salir

Opcion: 0

¡Hasta la próxima!
FIN DEL PROGRAMA

3.2 Situaciones excepcionales

El apartado anterior muestra el funcionamiento completo del programa, sin embargo no es una situación real, ya que he seguido el orden correcto al seleccionar las distintas opciones, he introducido en todos los menús opciones válidas y en su formato correcto. Un usuario normal es impredecible, ya que suelen cometer muchos fallos al navegar por el menú del programa, debido a que pueden no estar acostumbrados a usar este tipo de interfaces.

Para solventar este problema es necesario hacer un tratamiento de excepciones en todo el programa y en cada situación:

3.2.1 Tratamiento de selecciones dentro de los menús del programa: Cuando se muestran los menús del programa, se solicita al usuario que elija una de las opciones que aparecen dentro del mismo, y el programa necesita que la respuesta del usuario sea en formato numérico, pero el usuario no tiene por qué saberlo, por lo que se muestra un mensaje indicando al usuario que la opción introducida no es válida.

1- Iniciar sesion
2- Registrarse
0- Salir

Opcion: Uno

El valor introducido no es un numero

También existe una protección ante la posibilidad de que el usuario elija numeros negativos o que no se correspondan con ninguna de las opciones posibles.

```
1- Iniciar sesion
2- Registrarse
0- Salir

Opcion: -1
Introduce una opción válida (1, 2 o 0)
```

Este tratamiento de excepciones está aplicado a todos las opciones del programa, tanto con la sesión iniciada, como con la sesión cerrada.

3.2.2 Comprobación de credenciales de inicio de sesión: Si el usuario intenta iniciar sesión dentro del programa, necesita introducir un nombre de usuario y una contraseña, por lo que la primera acción que debe realizar el programa es la comprobación de la existencia del nombre de usuario. En caso de que el usuario no exista o la contraseña no coincida con el usuario introducido, el programa mostrará el siguiente mensaje de error y abortará el intento de inicio de sesión.

```
Opcion: 1
Nombre de usuario: Pepe12
Contraseña: pepe
El usuario y/o contraseña son incorrectos
Pulse 'enter' para continuar
```

3.2.3 Validaciones en el proceso de registro: Para que el registro de una cuenta nueva en base de datos se pueda llevar a cabo, es necesario validar que todos los datos que el usuario introduce, cumplan una serie de condiciones. Si se produce algún error en la introducción de los datos, debido a el incumplimiento de algun formato, o el dato ya está registrado dentro de la base de datos (en caso de tratarse de un **NIF**, **email**, o **username**) ocurrirá lo siguiente:

```
Opcion: 2
Introduce nombre de usuario: PepePerez3
Contraseña: 1
La contraseña no es válida (1 letra, 1 numero y 8 caracteres como mínimo)

Pulse 'enter' para continuar
```

El programa muestra un mensaje de error indicando al usuario donde se encuentra el fallo a la hora de registrarse, de manera que pueda intentarlo de nuevo desde cero, ya que cuando se produce un fallo el registro se solicita integramente de nuevo. En caso de que el error venga del intento de registro de un usuario (username) que esté ya en uso, se mostrará de la siguiente manera:

```
Opcion: 2
Introduce nombre de usuario: Danigr6
El nombre de usuario ya está en uso
Pulse 'enter' para continuar
```

Y como ocurre con estos dos ejemplos, todos los campos de registro (exceptuando la dirección) requieren ser rellenados y cumplir una serie de requisitos de formato.

3.2.4 Control de excepciones al tratar con laberintos: Una vez que pasamos al menú Logged del programa, es necesario avisar al usuario de que es necesario hacer ciertas cosas antes de intentar procesar un laberinto, mostrando mensajes de error que indiquen el orden correcto de selección de las opciones. Voy a simular las posibles situaciones que pueden ocurrir:

- **Mostrar laberinto sin cargar ningún mapa previamente.**

```
Opción: 2
No se ha cargado ningún laberinto
Pulse 'enter' para continuar
```

- **Buscar camino sin cargar ningún laberinto previamente.**

```
Opción: 4
NO se ha cargado ningún mapa
Pulse 'enter' para continuar
```

- **Establecer casillas sin cargar ningún mapa previamente.**

```
Opción: 3
No se ha cargado ningún laberinto
Pulse 'enter' para continuar
```

- **Buscar camino sin establecer casillas (Con mapa cargado).**

```
Opción: 4
Entrada y salida NO implementadas!!!
Pulse 'enter' para continuar
```

3.3.5 Establecimiento de casillas de entrada y salida: Cuando el usuario ya se ha decantado por un laberinto y se ha cargado completamente dentro del sistema, se le solicitarán las coordenadas de entrada y de salida, las cuales deben corresponder a una coordenada dentro del mapa y que esa coordenada dentro del mapa no corresponda con un muro.

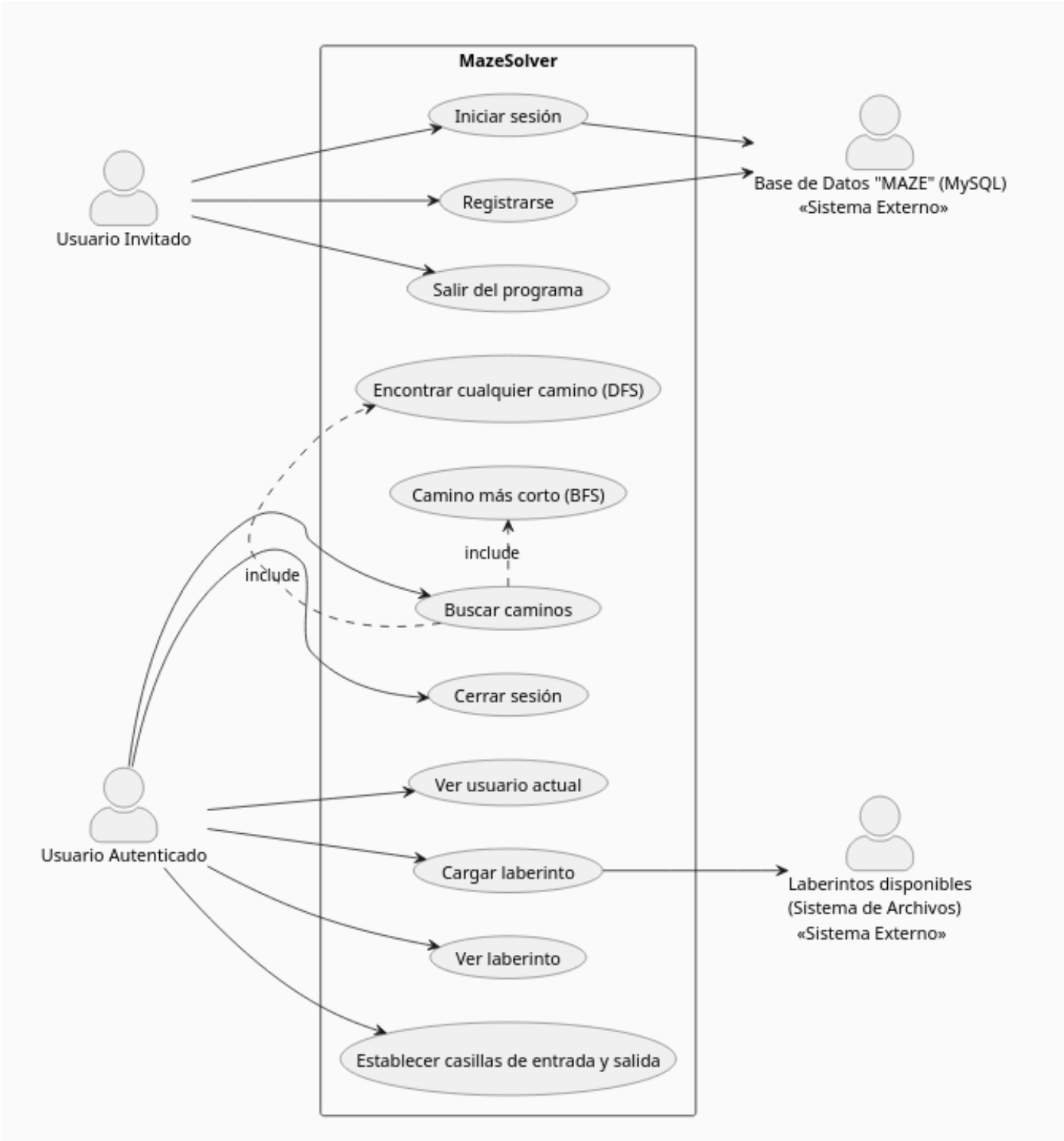
Para evitar problemas de ejecución de los algoritmos, es necesario estar seguro de que la coordenada escrita es una coordenada plenamente válida, por lo que se muestra un mensaje de error del sistema donde se solicita una nueva coordenada al usuario

1º Coincidencia con muro	Opción: 3 Para establecer la entrada indica las Valor de I: 0 Valor de J: 10 Las coordenadas coinciden con un muro
2º Valores no válidos	Valor de I: -1 Valor de J: 0 Valor no válido!!!
3º Valores no numéricos	Valor de I: -1 Valor de J: a ERROR: Introduce un valor numérico

4. Definición en UML del programa

4.1 Diagrama casos de uso

Para entender mejor la estructura interna del programa y cómo interactúan los usuarios con el sistema, he desarrollado un diagrama de casos de uso bajo el estándar UML. Con este modelo se hace más sencillo visualizar los diferentes actores que intervienen dentro de la aplicación y se definen las funcionalidades que el programa pone a disposición del usuario. A través de esta representación gráfica, se detallan las acciones principales que un usuario puede ejecutar, abarcando desde el proceso inicial de registro e inicio de sesión, hasta la carga y resolución automatizada de los laberintos mediante los diferentes algoritmos. De este modo, el diagrama sirve como un mapa conceptual que delimita el alcance funcional de esta versión del programa y aclara cómo el sistema responde a cada una de las peticiones del usuario antes de pasar al desarrollo del código.



4.1.1 Fichas casos de uso

Nombre	Inicio de sesión y autenticación de usuarios.
Actor	Usuario invitado.
Precondiciones	El usuario debe tener una cuenta dentro del sistema, por lo que debe estar previamente registrado.
Flujo principal	1. El usuario selecciona la opción 1 del menú inicial, correspondiente al inicio de sesión. 2. Se introducen las credenciales de acceso (nombre de usuario y contraseña). 3. La clase DAO encripta la contraseña en texto plano introducida por el usuario y realiza la comparación con el valor almacenado en la base de datos MySQL. 4. Si en la consulta de base de datos por parte del DAO, las credenciales coinciden con éxito, el estado de la sesión pasa a activo y se procede a instanciar un objeto User, para permitir acceder a todas las funcionalidades del programa.
Postcondiciones	El sistema habilita y despliega el menú con las opciones exclusivas para usuarios autenticados.
Excepciones	En caso de que el nombre de usuario no exista o la contraseña no coincida con la del usuario introducido, el sistema muestra un mensaje de error (El usuario y/o contraseña son incorrectos) por motivos de seguridad, evitando revelar qué dato concreto falló.

Nombre	Proceso de registro de nuevos usuarios.
Actor	Usuario invitado.
Precondiciones	Se requiere que el usuario actual no tenga una sesión activa en el sistema.
Flujo Principal	1. El usuario selecciona la segunda opción del menú inicial del programa, correspondiente al registro. 2. El sistema solicita el ingreso de los siguientes datos: nombre de usuario, contraseña, nombre completo, NIF, correo electrónico, dirección y fecha de nacimiento. 3. El sistema verifica la validez del formato de la información ingresada para asegurar la correcta creación del usuario dentro de la base de datos. 4. La clase DAO realiza una consulta para asegurar que el NIF, correo electrónico o nombre de usuario no se encuentren previamente registrados. 5. La contraseña introducida es encriptada mediante el algoritmo MD5, tras lo cual se procede a la consolidación definitiva del registro en la base de datos MySQL.
Postcondiciones	El proceso de registro se completa satisfactoriamente, regresando al usuario al menú inicial.
Excepciones	En caso de detectarse un formato inválido en los datos proporcionados o la existencia de registros duplicados para el nombre de usuario, correo electrónico o NIF, el proceso se interrumpe y aborta de inmediato.

Nombre	Selección, procesamiento y establecimiento de casillas de entrada y salida del laberinto
Actor	Usuario Autenticado.
Precondiciones	El usuario debe tener una sesión iniciada con éxito para poder acceder a las opciones de cargar, mostrar y establecer casillas del laberinto.
Flujo Principal	1. El usuario selecciona la opción 1 del menú de sesión iniciada, correspondiente a la elección del laberinto a procesar. 2. El usuario selecciona una de las opciones entre los diferentes archivos txt de laberintos disponibles. 3. El sistema abre el archivo seleccionado y lee el archivo del laberinto seleccionado, convirtiéndolo en una matriz de char local que posteriormente será procesada. 4. El usuario selecciona la opción 3 correspondiente al establecimiento de las casillas de entrada y salida del laberinto, donde introducirá las coordenadas deseadas.
Postcondiciones	El mapa se carga correctamente en la memoria del programa.
Excepciones	En caso de que las coordenadas no sean válidas (se salen de los rangos de la matriz) o coincidan con un muro, el programa solicitará nuevas coordenadas.

Nombre	Mostrar laberinto actual
Actor	Usuario Autenticado.
Precondiciones	El usuario debe tener una sesión iniciada con éxito, donde se haya cargado un laberinto en memoria.
Flujo Principal	1. El usuario selecciona la opción 2 que corresponde a ver el laberinto actual. 2. El programa procesa el laberinto cargado dentro del sistema y sustituye los caracteres de texto plano por emojis representativos, dando una imagen más visual del laberinto. 3. En caso de que el usuario haya establecido previamente las casillas de entrada y salida, se muestran también en la imagen..
Postcondiciones	La sucesión de coordenadas queda registrada dentro del programa, permitiendo ver la solución del algoritmo en cualquier momento (a menos que se solicite otra ruta).
Excepciones	El usuario que entra en la opción de selección de laberinto no introduce una opción válida o selecciona la opción 0, que vuelve al menú anterior.

Nombre	Resolución de laberintos
Actor	Usuario Autenticado.
Precondiciones	El usuario debe tener una sesión iniciada con éxito, donde se haya cargado un laberinto en memoria y se hayan establecido las casillas de entrada y salida.

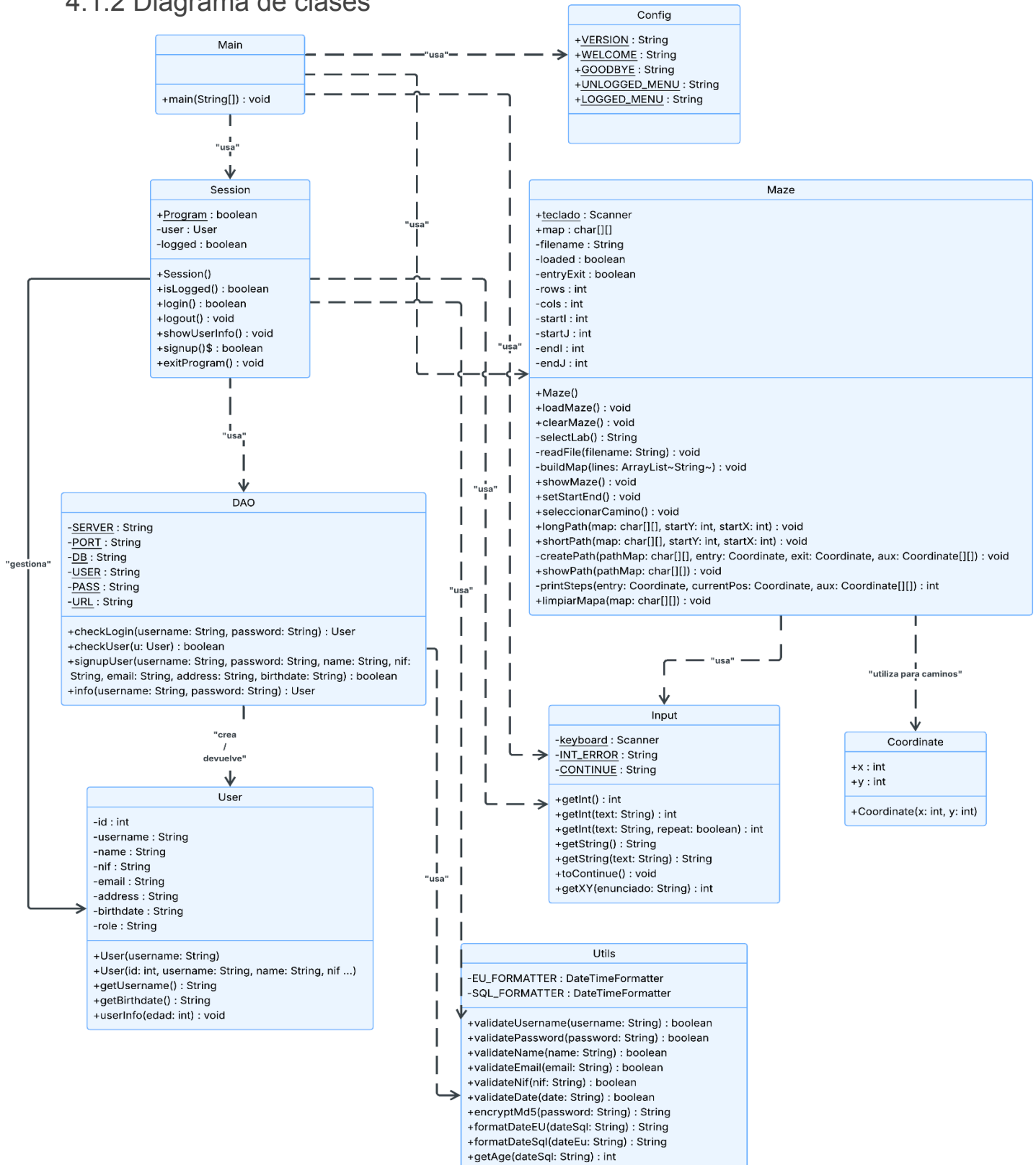
Flujo Principal	1. El usuario selecciona la opción 4 que corresponde con la funcionalidad “Buscar caminos” del programa. 2. El usuario elige una de las dos posibilidades de resolución del laberinto cargado, la opción larga con el algoritmo DFS o la opción corta con el algoritmo BFS. 3. El sistema ejecuta el algoritmo seleccionado creando una ruta sobre la matriz del mapa.. 4. El programa termina el proceso de creación de la ruta, y muestra el resultado de forma visual, marcando cada coordenada de cada paso del camino y haciendo el recuento final de los pasos necesarios para completar el laberinto.
Postcondiciones	La sucesión de coordenadas queda registrada dentro del programa, permitiendo ver la solución del algoritmo en cualquier momento (a menos que se solicite otra ruta).
Excepciones	El usuario que entra en la opción de selección de laberinto no introduce una opción válida o selecciona la opción 0, que vuelve al menú anterior.

Nombre	Cierre de sesión
Actor	Usuario Autenticado.
Precondiciones	El usuario debe tener una sesión iniciada con éxito.

Flujo Principal	1. El usuario selecciona la opción 4 que corresponde con la funcionalidad cerrar sesión. 2. Internamente, se llama al método logout para cambiar el booleano logged de true a false, eliminando también el objeto User y vaciando la memoria del mapa. 3. El programa devuelve al usuario al menú de inicio del programa.
Postcondiciones	El usuario vuelve a su estado inicial, ya que se elimina toda la información relativa a la sesión cerrada, de manera que se protege la integridad de sus datos.
Excepciones	No existen excepciones en este proceso.



4.1.2 Diagrama de clases



5. Definición individual de cada clase del programa

5.1 Clase Main

La clase Main actúa como el punto de entrada oficial de la aplicación. Su propósito principal es inicializar los componentes del sistema (gestión de sesión y el laberinto) y controlar el bucle principal de ejecución, derivando el flujo a los diferentes menús interactivos según el estado de la autenticación del usuario.

5.1.1 Atributos de la Clase

La clase Main no posee atributos de clase ni variables.

5.1.2 Métodos y Operaciones

A continuación se detalla la firma, visibilidad y comportamiento del método que conforma la clase:

```
public static void main(String[] args)
```

Descripción: Imprime por pantalla el mensaje de bienvenida y evalúa las opciones ingresadas por consola mediante una estructura condicional anidada. Es la clase encargada de gestionar todo el funcionamiento del programa y las llamadas a metodos principales dentro de las clases Maze y Session.

Flujo de Control:

1º Si el usuario no está logueado (session.isLoggedIn() == false), despliega las opciones de inicio de sesión, registro o salida empleando constantes de Config e interacciones de Input.

2º Si el usuario se encuentra autenticado, conmuta automáticamente al menú del laberinto (carga, visualización, establecimiento de coordenadas, resolución de caminos o cierre de sesión).

Excepciones controladas: Captura y gestiona de forma controlada la excepción FileNotFoundException en caso de fallos críticos en la lectura de ficheros del sistema de laberintos.

5.2 Clase Session

La clase Session es el componente encargado de gestionar la seguridad, el estado de autenticación de los usuarios y el ciclo de vida de la ejecución del programa. Centraliza los mecanismos de control de acceso, la validación de credenciales a través del acceso a datos, y la captura de la información del perfil del usuario activo.

5.2.1 Atributos de la Clase

- **public static boolean Program:** Booleano global inicializado en true, que determina si el bucle principal de la aplicación en la clase Main debe continuar ejecutándose o finalizar de forma definitiva.
- **private static User user:** Instancia estática de la clase User, encargada de almacenar y representar al usuario que ha iniciado sesión correctamente en el sistema, permitiendo el acceso global a sus datos de perfil.
- **private boolean logged:** Atributo que sirve como indicador del estado de la sesión actual para controlar el acceso del cliente (true si está autenticado, false si no lo está).

5.2.2 Métodos y Operaciones

public Session()	Descripción: Constructor de la clase. Inicializa por defecto el estado de la sesión (logged) en false cada vez que se arranca la aplicación.
public boolean isLoggedIn()	Descripción: Método que devuelve el estado del atributo logged.
public boolean login()	Descripción: Gestiona el proceso de autenticación. Solicita mediante consola el nombre de usuario y la contraseña empleando la clase Input, cifra la clave en MD5 invocando a Utils y realiza la verificación en la base de datos a través de una instancia local de la clase DAO. Si las credenciales son válidas, establece el booleano logged a true, recuperando la información completa del usuario.
public void logout()	Descripción: Restablece los parámetros de sesión al estado inicial. Modifica el atributo logged a false y establece el objeto estático user a un valor nulo (null), garantizando el borrado seguro en memoria de los datos del usuario actual.
public void showUserInfo()	Descripción: Controla la visualización del perfil en pantalla. Si existe un usuario activo en el sistema (user != null), procesa de forma anidada el cálculo de la edad actual a través de las funciones de parseo de fechas de la clase Utils y muestra la información estructurada del usuario a través del método userInfo() de la clase User.

<code>public static boolean signup()</code>	Descripción: Método estático encargado de registrar nuevos usuarios. Recoge secuencialmente los datos requeridos por consola y los somete a rigurosas expresiones regulares de validación (Utils). Si la información es válida, encripta la contraseña, formatea la fecha al estándar SQL e instruye a la clase DAO para realizar la inserción.
<code>public void exitProgram()</code>	Descripción: Finaliza de manera controlada la ejecución global de la aplicación modificando la propiedad estática Program a false.

5.3 Clase User

La clase User ha sido diseñada para representar la estructura de datos de un usuario dentro del sistema. Su propósito principal es encapsular toda la información del usuario recuperada de la base de datos a través de variables de instancia de carácter privado. De este modo, no solo actúa como un contenedor de datos seguro durante la sesión activa, sino que además provee la lógica interna necesaria para formatear y proyectar de manera limpia el perfil del usuario en la interfaz de la consola.

5.3.1 Atributos de la Clase

Todos los atributos de esta clase son privados (`private`) para cumplir estrictamente con el principio de encapsulamiento de la programación orientada a objetos:

- **private int id:** Identificador único y numérico del usuario, correspondiente a la clave primaria (PK) en la base de datos relacional.
- **private String username:** Nombre de cuenta único elegido por el usuario para efectuar sus credenciales de acceso.
- **private String name:** Nombre y apellidos completos del titular de la cuenta.
- **private String nif:** Número de Identificación Fiscal (DNI/NIE) asociado al usuario.
- **private String email:** Dirección de correo electrónico de la cuenta.
- **private String address:** Dirección física o domicilio residencial del usuario.
- **private String birthdate:** Fecha de nacimiento del usuario, almacenada temporalmente en formato de texto.
- **private String role:** Rol o nivel de privilegios asignado a la cuenta dentro del sistema.

5.3.2 Métodos y Operaciones

<code>public User(String username)</code>	Descripción: Constructor que se encarga de instanciar un objeto de usuario simplificado utilizando únicamente el nombre de cuenta como parámetro de inicialización.
<code>public User(int id...)</code>	Descripción: Constructor completo parametrizado. Se encarga de mapear e inicializar cada uno de los atributos de la clase con los registros de datos procesados y devueltos por el sistema de persistencia de datos (DAO).
<code>public String getUsername()</code>	Descripción: Método que devuelve el valor del atributo privado username.
<code>public String getBirthdate()</code>	Descripción: Método que devuelve la fecha de nacimiento guardada en el atributo privado birthdate.
<code>public void userInfo(int edad)</code>	Descripción: Método encargado de la salida de datos por consola. Imprime de forma tabular y estructurada en pantalla todo el perfil completo del usuario activo (ID, nombre, NIF, email, dirección y rol), adjuntando además el parámetro numérico de edad calculado externamente.

5.4 Clase Maze

La clase Maze constituye el núcleo algorítmico y operacional del programa. Su responsabilidad principal es la carga, representación bidimensional, manipulación y resolución de laberintos. Implementa estructuras de datos matriciales para almacenar el mapa y encapsula los algoritmos de búsqueda y exploración para calcular tanto el camino más corto como rutas alternativas entre las coordenadas de entrada y salida especificadas por el usuario.

5.4.1 Atributos de la Clase

- **public static Scanner teclado:** Instancia estática del objeto Scanner compartida a nivel de clase para gestionar la lectura de flujos de entrada de datos específicos del laberinto por consola.
- **public char[][] map:** Matriz bidimensional de caracteres que almacena físicamente la topología del laberinto en memoria (muros, pasillos, entradas y salidas).
- **private String filename:** Almacena el nombre o la ruta del archivo de texto (.txt) del laberinto.

- **private boolean loaded:** Indicador que determina si un mapa ha sido cargado con éxito en la memoria del programa.
- **private boolean entryExit:** Indicador booleano que registra si el usuario ya ha definido de forma válida las casillas de inicio y fin dentro del mapa actual.
- **private int rows, cols:** Variables enteras que guardan las dimensiones dimensionales (filas y columnas) extraídas del fichero del laberinto.
- **private int startI, startJ:** Coordenadas cartesianas matriciales (fila y columna) correspondientes al punto de entrada (I) del laberinto.
- **private int endI, endJ:** Coordenadas cartesianas matriciales (fila y columna) correspondientes al punto de destino o salida (F).

5.4.2 Métodos y Operaciones

public Maze()	Descripción: Constructor de la clase. Inicializa por defecto los índices de las posiciones de entrada y salida en -1, indicando que aún no se han configurado en el sistema.
public void loadMaze()	Descripción: Gestiona el proceso de inicialización del mapa. Invoca al selector de archivos, activa la lectura del fichero físico y actualiza el estado del booleano loaded a true restableciendo la configuración de entradas previas.
public void clearMaze()	Descripción: Libera los recursos de memoria asignando un valor nulo (null) a la estructura de la matriz y revierte todas las propiedades lógicas y de ruta a sus estados iniciales por defecto.
private String selectLab()	Descripción: Método privado auxiliar. Escanea el directorio interno del sistema destinado a almacenar los laberintos (./assets/mazes/), lista los ficheros disponibles y permite al usuario elegir dinámicamente cuál cargar.

private void readFile (String filename)	Descripción: Es el método encargado de la lectura física y procesamiento línea por línea del archivo de texto (.txt) que contiene el laberinto seleccionado. Utiliza un objeto File asociado a la ruta del archivo y un flujo de lectura estructurado mediante la clase Scanner.
private void buildMap (ArrayList<String> lines)	Descripción: Procesa e interpreta las cadenas de texto del archivo para instanciar la matriz map[[]]. Determina las dimensiones físicas (rows y cols) y asigna a cada celda el carácter representativo correspondiente (# para muros, " " para espacios vacíos).
public void showMaze()	Descripción: Proyecta visualmente el laberinto en la terminal del sistema. Recorre de forma bidimensional la matriz y traduce de forma estética cada carácter ASCII a un emoticono gráfico legible para el usuario final (muros como bloques de ladrillo, el inicio como un ratón, el destino como un queso).
public void setStartEnd()	Descripción: Solicita interactivamente al usuario las posiciones espaciales por consola para fijar la entrada y salida. Valida que las coordenadas se encuentren dentro de los límites de la matriz y que no colisionen con un muro (#). Tras la validación, inserta los caracteres I y F en el mapa.
public void seleccionarCamino()	Descripción: Despliega el submenú de resoluciones y bifurca el flujo de control del programa según la preferencia del cliente (búsqueda del camino más óptimo o exploración extendida).
public void shortPath (char[][] map, int startY, int startX)	Descripción: Implementa el algoritmo de resolución del camino más corto (basado una búsqueda en anchura o BFS). Emplea estructuras de colas (Queue/LinkedList) y una matriz auxiliar de objetos Coordinate para trazar la ruta de menor recorrido desde el origen hasta el destino.
public void longPath (char[][] map, int startY, int startX)	Descripción: Explora los caminos del laberinto utilizando un algoritmo de búsqueda en profundidad (DFS). Es menos eficiente ya que no garantiza la ruta mínima y procesa más celdas redundantes antes de hallar la salida.

private void createPath (char[][] pathMap, Coordinate entry, Coordinate exit, Coordinate[][] aux)	Descripción: Reconstruye de forma inversa el camino resolutivo óptimo recorriendo la matriz auxiliar de punteros aux desde el nodo final de salida hacia el nodo origen, marcando el trayecto en el mapa con símbolos de dirección (<, >, ^, V).
public void showPath (char[][] pathMap)	Descripción: Imprime en la consola el mapa resultante que contiene la resolución gráfica del camino recorrido, utilizando iconos de flechas de dirección para ilustrar el sentido del flujo.
private int printSteps (Coordinate entry, Coordinate currentPos, Coordinate[][] aux)	Descripción: Algoritmo que detalla textualmente los pasos y movimientos requeridos (Arriba, Abajo, Izquierda, Derecha), devolviendo el coste total de pasos.
public void limpiarMapa (char[][] map)	Descripción: Vacía los caracteres de control generados por los algoritmos de búsqueda para restaurar el mapa original.

5.5 Clase Input

La clase Input es un componente de utilidad diseñado para centralizar, simplificar y asegurar la entrada de datos por consola. Encapsula una instancia de la clase Scanner y provee métodos estáticos sobrecargados que gestionan la lectura de cadenas y números enteros. Su función principal es interceptar mediante bloques try-catch cualquier entrada inválida (como introducir letras cuando se espera un número), evitando que el programa sufiera excepciones en tiempo de ejecución (runtime) y forzando la repetición del bucle de captura hasta que el usuario introduzca un dato correcto.

5.5.1 Atributos de la Clase

Todos los atributos de esta clase son privados y estáticos, actuando como recursos globales internos:

- **private static Scanner keyboard:** Instancia de la clase Scanner utilizada para escanear el flujo de entrada estándar (System.in) de la terminal.
- **private static final String INT_ERROR:** Constante de texto que almacena el mensaje estándar de error de tipado para advertir al usuario cuando introduce un valor no numérico.
- **private static final String CONTINUE:** Constante de texto que almacena el mensaje instructivo para pausar la pantalla y solicitar la pulsación de la tecla Intro.

5.5.2 Métodos y Operaciones

<code>public static int getInt()</code>	Descripción: Solicita un número entero de forma directa por consola. Si la entrada no es un entero válido, captura la excepción genérica, muestra el mensaje INT_ERROR y devuelve un valor -1.
<code>public static int getInt (String text)</code>	Descripción: Variante sobrecargada que imprime un enunciado previo (text) en la terminal antes de efectuar la lectura del número entero.
<code>public static int getInt (String text, boolean repeat)</code>	Descripción: Implementa un bucle condicional do-while que fuerza la repetición de la entrada de datos mientras el valor provisto por el usuario no se pueda parsear a entero con éxito. Si el parámetro repeat se configura en false y salta un error, aborta inmediatamente devolviendo -1.
<code>public static String getString()</code>	Descripción: Lee una línea completa de texto desde la consola y aplica el método .trim() para eliminar de forma automática espacios en blanco redundantes al inicio y al final de la cadena antes de devolverla.
<code>public static String getString (String text)</code>	Descripción: Variante sobrecargada que muestra un mensaje o etiqueta guía en pantalla antes de capturar y limpiar la cadena de texto del usuario.
<code>public static void toContinue()</code>	Descripción: Pausa la ejecución del flujo del programa imprimiendo la constante CONTINUE y queda a la espera de que el usuario pulse la tecla Intro para reanudar el menú.
<code>public static int getXY (String enunciado)</code>	Descripción: Método especializado diseñado para la captura individual de las coordenadas del laberinto. Imprime la instrucción en pantalla y, si se detecta una entrada inválida, imprime un mensaje de aviso específico en lugar del error genérico, evitando que la aplicación colapse.

5.6 Clase DAO

La clase DAO (Data Access Object) representa la capa de persistencia de la aplicación. Su responsabilidad exclusiva es centralizar el acceso a la base de datos relacional (MySQL) encapsulando toda la lógica de bajo nivel del API JDBC. Utiliza objetos estructurales como Connection, PreparedStatement y ResultSet para ejecutar de forma segura sentencias SQL parametrizadas, mitigando vulnerabilidades de inyección de código. Su función es actuar como un puente traductor que lee registros de las tablas relacionales y los transforma en objetos de la lógica de negocio (User) para su uso en el sistema.

5.6.1 Atributos de la Clase

Todos los atributos de esta clase son constantes privadas de instancia (private final) que definen las propiedades estrictas de conectividad con el servidor de base de datos:

- **private final String SERVER, PORT, DB, USER, PASS:** Almacenan respectivamente la dirección IP del servidor físico, el puerto de escucha del servicio, el nombre del esquema de la base de datos, el usuario administrador y su clave de acceso.
- **private final String URL:** Almacena la cadena de conexión unificada con la sintaxis del protocolo nativo del conector (jdbc:mysql://...) construida dinámicamente mediante concatenación en tiempo de inicialización.

5.6.2 Métodos y Operaciones

public User checkLogin (String username, String password)	Descripción: Realiza una consulta selectiva rápida en la base de datos para verificar la existencia de una cuenta con las credenciales introducidas. Si el registro coincide, inicializa y devuelve un objeto User básico con sus campos rellenados directamente desde las columnas obtenidas en el ResultSet.
public boolean checkUser (User u)	Descripción: Método de validación interna que comprueba si un nombre de usuario en concreto ya se encuentra registrado y ocupado dentro de la tabla de la base de datos, sirviendo de control preventivo antes de autorizar un nuevo registro.
public boolean signupUser (String username, String password, String name, String nif, String email, String address, String birthdate)	Descripción: Ejecuta una sentencia de inserción estructurada (INSERT INTO) para añadir de forma persistente un nuevo registro de usuario en el sistema. Emplea PreparedStatement para asignar de manera segura cada parámetro de texto, tratando los campos vacíos u opcionales (como address) con valores nulos (setNull) si es necesario. Devuelve true si la fila fue afectada con éxito en el servidor.

**public User info(String
username, String password)**

Descripción: Método especializado encargado de extraer la ficha técnica completa del perfil de un usuario tras un login exitoso. Recupera de forma íntegra todos los registros del cliente (ID, nombre, NIF, email, dirección, rol) e invoca internamente a las funciones estáticas de Utils para parsear y dar formato europeo (dd/MM/yyyy) a la fecha de nacimiento antes de instanciar y devolver el objeto User.

5.7 Clase Utils

La clase Utils es un componente de utilidad cuya única responsabilidad es proveer un conjunto de herramientas y funciones de soporte globales para el resto de los módulos de la aplicación. Al carecer de estado propio y no requerir instanciación, todos sus miembros están definidos como estáticos. Su lógica interna encapsula operaciones críticas de infraestructura informática, tales como la validación rigurosa de datos de entrada mediante expresiones regulares (Regex), el formateo de objetos cronológicos entre los estándares europeos y de persistencia relacional, y la seguridad criptográfica de contraseñas.

5.7.1 Atributos de la Clase

Todos los atributos de esta clase son privados, estáticos y constantes (`private static final`), sirviendo de patrones inmutables de configuración interna:

- **`private static final DateTimeFormatter EU_FORMATTER`:** Patrón de formateo cronológico configurado bajo la máscara europea tradicional (dd/MM/yyyy).
- **`private static final DateTimeFormatter SQL_FORMATTER`:** Patrón de formateo cronológico adaptado al estándar nativo ISO de las bases de datos relacionales (yyyy-MM-dd).

5.7.2 Métodos y Operaciones

**public static boolean
validateUsername
(String username)**

Descripción: Evalúa la cadena mediante una expresión regular que exige que la cuenta comience obligatoriamente por una letra y posea una longitud de entre 3 y 15 caracteres alfanuméricos o de control (., _).

**public static boolean
validatePassword(String
password)**

Descripción: Evalúa la cadena mediante una expresión regular para garantizar que la clave contenga al menos una letra y un dígito numérico, con una extensión mínima de 8 caracteres.

public static boolean validateName(String name)	Descripción: Valida que la cadena corresponda a un formato nominal compuesto (nombre y apellidos) permitiendo caracteres alfabéticos acentuados y con espacios intermedios obligatorios.
public static boolean validateEmail(String email)	Descripción: Valida que la cadena corresponda a una dirección de correo con presencia del identificador de usuario, el símbolo @ y un dominio lícito.
public static boolean validateNif(String nif)	Descripción: Verifica la sintaxis del documento nacional de identidad o NIE mediante una expresión regular adaptada a la estructura oficial del estado.
public static boolean validateDate(String date)	Descripción: Intenta parsear la fecha ingresada bajo el patrón europeo; si se arroja una anomalía de tiempo, la captura y determina que la fecha es inválida cronológicamente.
public static String encryptMd5(String password)	Descripción: Transforma la contraseña en texto plano en un resumen criptográfico de 32 caracteres hexadecimales utilizando el algoritmo hash MD5 nativo de MessageDigest. Si ocurre un fallo en el entorno Java, propaga una excepción en tiempo de ejecución.
public static String formatDateEU(String dateSql)	Descripción: Toma una cadena en formato de base de datos (yyyy-MM-dd) y la transfiere al formato legible europeo (dd/MM/yyyy). Captura posibles errores de parseo devolviendo un mensaje de control.
public static String formatDateSql(String dateEu)	Descripción: Realiza la operación inversa de formateo. Reemplaza preventivamente guiones por barras e interpreta la cadena europea para devolver la sintaxis adecuada que requiere la base de datos.
public static int getAge (String dateSql)	Descripción: Calcula la edad exacta en años basándose en el registro de nacimiento. Convierte la cadena en una fecha del sistema (LocalDate) y determina el diferencial temporal transcurrido con respecto a la fecha actual del servidor (LocalDate.now()) por medio de la clase Period.

5.8 Clase Coordinate

La clase `Coordinate` representa una entidad de datos ligera cuyo único propósito es modelar un punto o posición espacial bidimensional (X, Y) dentro de la matriz del laberinto. Actúa como un contenedor estructurado de coordenadas cartesianas, facilitando la instanciación de posiciones y el almacenamiento ordenado de rutas o casillas. Es un componente fundamental para que los algoritmos de búsqueda y resolución de la clase `Maze` puedan rastrear el camino, encolar nodos visitados y almacenar los punteros de retorno en las matrices auxiliares.

5.8.1 Atributos de la Clase

Al tratarse de una clase puramente de datos utilizada para operaciones matriciales directas, sus atributos se definen con visibilidad pública para agilizar el acceso algorítmico sin sobrecargar la memoria con métodos selectores repetitivos:

- **public int x:** Variable entera que representa la posición en el eje horizontal (columna) dentro de la cuadrícula del laberinto.
- **public int y:** Variable entera que representa la posición en el eje vertical (fila) dentro de la cuadrícula del laberinto.

5.8.2 Métodos y Operaciones

<code>public Coordinate (int x, int y)</code>	Descripción: Constructor único parametrizado de la clase. Se encarga de recibir los dos índices numéricos enteros correspondientes a la ubicación espacial e inicializar de forma directa las propiedades <code>x</code> e <code>y</code> del objeto en el momento de su creación.
---	---

5.9 Clase Config

La clase `Config` es un componente de utilidad diseñado con el único propósito de centralizar y gestionar todas las constantes globales de texto de la aplicación. Se define como una clase estática abstracta en su concepción (no requiere instanciación). Su función principal en la arquitectura es actuar como un repositorio unificado de cadenas de caracteres, facilitando el mantenimiento y las futuras modificaciones de los textos informativos, mensajes de despedida y los diferentes menús interactivos del sistema sin necesidad de alterar el código fuente de los flujos de control.

5.9.1 Atributos de la Clase

Todos los atributos de esta clase son públicos, estáticos y constantes (`public static final`), lo que permite que cualquier otro módulo del programa los invoque de manera directa:

- **public static final String VERSION:** Almacena de manera inmutable el identificador de la versión de desarrollo actual del programa (v1.1.0).

- **public static final String WELCOME:** Guarda el mensaje institucional de bienvenida del programa, asociando la autoría del estudiante de DAM.
- **public static final String GOODBYE:** Contiene la cadena informativa de cierre que se proyecta al salir de forma definitiva de la aplicación.
- **public static final String UNLOGGED_MENU:** Estructura el texto y las opciones numéricas del menú principal (Login, Registro, Salir) destinado a los usuarios anónimos o no autenticados en el sistema.
- **public static final String LOGGED_MENU:** Modula el árbol de opciones y funcionalidades del sistema del laberinto (Cargar, Ver, Coordenadas, Resolución, Perfil de Usuario, Cierre de sesión) exclusivo para clientes que han superado el login.

5.9.2 Métodos y Operaciones

La clase Config no posee funciones, comportamientos ni métodos operacionales internos al tratarse de una clase puramente de definición y configuración de datos constantes.

6. Desarrollo de la implementación de cada funcionalidad individual del programa

Este apartado describe detalladamente el comportamiento técnico, el flujo de control y las interconexiones de los módulos del sistema al ejecutar cada una de las opciones operacionales provistas en las interfaces del programa.

6.1 Funcionalidades en Estado No Autenticado

6.1.1 Iniciar Sesión (Login)

Descripción Técnica: Permite la autenticación de usuarios preexistentes para habilitar las opciones de explotación de laberintos.

Flujo de Implementación:

1. El hilo de ejecución invoca de manera secuencial a `Input.getString(text)` para capturar por consola el nombre de cuenta y la contraseña en texto plano.
2. La cadena de la contraseña se transfiere inmediatamente a `Utils.encryptMd5(password)`, la cual aplica un algoritmo hash criptográfico para generar una firma irreversible de 32 caracteres hexadecimales.

3. Se instancia un flujo hacia la capa de persistencia mediante el método `.checkLogin` (username, password) de la clase DAO. Este objeto abre una conexión con el servidor MySQL y ejecuta una consulta preparada (PreparedStatement) parametrizada para evitar inyecciones SQL.

4. Si la consulta devuelve un conjunto de datos válido, se crea una instancia básica del objeto User y se establece el atributo booleano de instancia logged de la clase Session a true. Posteriormente, se invoca `.info()` del DAO, para obtener el resto de los atributos del perfil.

- **Tratamiento de Excepciones:** Si ocurre una desconexión o fallo en la BBDD, el bloque interno try-catch de la capa de datos captura la excepción y la reporta de manera controlada por consola sin colapsar el hilo de ejecución principal.

```
public boolean login() {
    try {
        DAO connection = new DAO();
        String username = Input.getString("Nombre de usuario: ");
        String password = Input.getString("Contraseña: ");
        String passMd5 = Utils.encryptMd5(password);

        User u = connection.checkLogin(username, passMd5);
        if (u != null) {
            logged = true;
            user = connection.info(username, passMd5);
            return true;
        }
    } catch (Exception e) {
        System.out.println("Error: No se pudo procesar el login. " + e.getMessage());
    }

    System.out.print("El usuario y/o contraseña son incorrectos");
    return false;
}
```

6.1.2 Registro (Signup)

Descripción Técnica: Permite el proceso registro para nuevos usuarios, garantizando la integridad de los datos mediante expresiones regulares.

Flujo de Implementación:

1. El método estático `.signup()` de la clase Session coordina la captura guiada de datos (usuario, contraseña, nombre, NIF, email, domicilio y fecha de nacimiento) utilizando la clase Input.
2. Cada valor es comprobado por medio de los métodos de validación de Utils (validateUsername, validatePassword, validateEmail, etc.), los cuales contrastan las cadenas de texto frente a máscaras estructuradas de expresiones regulares (Regex).

3. Para el campo de la fecha, `Utils.formatDateSql(dateEu)` procesa la cadena de fecha europea (dd/MM/yyyy) y la reformatea al estándar ISO internacional (yyyy-MM-dd) requerido por la base de datos.
4. Tras validar el bloque completo, se invoca `.checkUser()` de la clase DAO para certificar que el identificador de cuenta no esté duplicado. Tras la comprobación se ejecuta el método `.signupUser()` de la clase DAO, enviando la clave tras ser tratada con un hash MD5 mediante `Utils.encryptMd5()`. El driver JDBC ejecuta una sentencia INSERT INTO en la tabla de usuarios de MySQL.

```
public static boolean signup() throws FileNotFoundException {
    DAO connection = new DAO();
    String username = Input.getString("Introduce nombre de usuario: ").trim();
    if(!Utils.validateUsername(username)) {
        System.out.println("Nombre no válido!!(Debe empezar por letra, 3-16 caracteres)");
        return false;
    }
    User u = new User(username);
    if (connection.checkUser(u)) {
        System.out.println("El nombre de usuario ya está en uso");
        return false;
    }

    String password = Input.getString("Contraseña: ").trim();

    if(!Utils.validatePassword(password)) {
        System.out.println("La contraseña no es válida (1 letra, 1 numero y 8 caracteres como mínimo)");
        return false;
    }

    String confirmPass = Input.getString("Confirma la contraseña: ").trim();
    if(!password.equals(confirmPass)) {
        System.out.println("La contraseña no coincide!!");
        return false;
    }
}
```

6.1.3 Cerrar programa

Descripción Técnica: Finaliza la ejecución del software de forma limpia cuando el usuario se encuentra en la pantalla de bienvenida.

Flujo de Implementación:

Al procesar la opción numérica correspondiente, la clase Main deriva la operación hacia el método `Session.exitProgram()`. Este procedimiento modifica el valor de la propiedad booleana `Session.Program` a `false`. El cambio de valor en el booleano `Program` permite que el bucle condicional `while` de la clase principal finalice, imprimiendo la constante de texto `Config.GOODBYE` y cerrando los hilos del proceso de la máquina virtual Java.

```
public void exitProgram() {
    Program = false;
}
```

6.2 Funcionalidades en Estado Autenticado

6.2.1 Cargar Laberinto

Descripción Técnica: Permite al usuario seleccionar un archivo de texto (.txt) con la estructura del laberinto para procesarlo y cargarlo como una matriz bidimensional en la memoria de la aplicación.

Flujo de Implementación:

1. Se invoca el método privado auxiliar `Maze.selectLab()`, el cual explora el directorio local de almacenamiento interno (`./assets/mazes`). Lista los nombres de los archivos legibles en la terminal y solicita una selección numérica mediante `Input.getInt()`.
2. Una vez obtenida la ruta del archivo, se ejecuta `Maze.readFile(filename)` para asociar el fichero a un objeto `File` y abrir un flujo de entrada mediante `Scanner` que permita leer su contenido.
3. El método procesa el archivo fila por fila, incorporando cada cadena analizada dentro de una lista dinámica de tipo `ArrayList<String>`.
4. Los datos pasan a `Maze.buildMap(lines)`. Este bloque calcula dinámicamente el número de filas y columnas del laberinto para instanciar la matriz bidimensional `map[][]` y poblarla con los caracteres que delimitan los pasillos (espacios en blanco) y los muros (`#`). Tras completar este proceso, se cambia el estado del indicador `loaded` a `true`.

Tratamiento de Excepciones: El proceso de lectura está protegido internamente con un bloque `try-catch` que intercepta la excepción nativa `FileNotFoundException`. Si el archivo seleccionado es inaccesible, el sistema emite una alerta textual informativa por consola, manteniendo la aplicación estable.

```
public void loadMaze() {  
    filename = selectLab();  
    if (filename != null) {  
        readFile(filename);  
        loaded = true;  
        entryExit = false;  
        System.out.print("\nMapa cargado correctamente!!\n");  
    }  
}
```

6.2.2 Ver Laberinto

Descripción Técnica: Proyecta de manera gráfica en la interfaz de usuario la topología y el estado actual del mapa cargado en la memoria de la aplicación.

Flujo de Implementación: El método `Maze.showMaze()` evalúa en primera instancia la el booleano `loaded`. Si su estado es falso, deniega la operación advirtiéndole al usuario de que debe inicializar un mapa. En caso afirmativo, se inicia un algoritmo de recorrido bidimensional anidado (mediante bucles combinados de índices `j` e `i`) para procesar las posiciones de la matriz `map[][]`. En cada celda analizada, una estructura de decisión traduce el carácter ASCII interno a emoticonos para el usuario final (los muros `#` se proyectan como bloques, el inicio `I` como un ratón y el destino `F` como un queso), garantizando una visualización clara y estilizada en la terminal.

```
public void showMaze() {
    if (!loaded) {
        System.out.print("\nNo se ha cargado ningún laberinto");
        return;
    }

    String[] numerosE = { "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11",
        "12", "13", "14", "15", "16", "17", "18", "19", "20", "21",
        "22", "23", "24", "25", "26", "27", "28", "29", "30", "31",
        "32", "33", "34", "35", "36", "37", "38", "39", "40", "41",
        "42", "43", "44", "45", "46", "47", "48", "49", "50" };

    System.out.print(" = ");
    for (int j = 0; j < cols; j++) {
        if (j >= 10) {
            System.out.print(numerosE[j / 10]);
        } else {
            System.out.print(" ");
        }
    }
    System.out.println();
    System.out.print(" = ");
    for (int j = 0; j < cols; j++) {
        System.out.print(numerosE[j % 10]);
    }
    System.out.println();

    for (int i = 0; i < rows; i++) {
        if (i < 10) {
            System.out.print(" = " + numerosE[i]);
        } else {
            System.out.print(numerosE[i]);
        }
        for (int j = 0; j < cols; j++) {
            if (map[i][j] == '#') {
                System.out.print("■");
            } else if (map[i][j] == 'I') {
                System.out.print("🐭");
            } else if (map[i][j] == 'F') {
                System.out.print("🧀");
            } else {
                System.out.print(" ");
            }
        }
        System.out.println();
    }
}
```

6.2.3 Establecer Coordenadas de Entrada y Salida

Descripción Técnica: Permite el establecimiento de las coordenadas de inicio y fin sobre el mapa para que los algoritmos de búsqueda puedan operar.

Flujo de Implementación:

1. El método `Maze.setStartEnd()` invoca de manera iterativa y dirigida a la función especializada `Input.getXY()` para capturar de forma individual las coordenadas (x, y).
2. El sistema somete los valores capturados a dos filtros lógicos de validación: primero, comprueba que los índices no superen los límites físicos de las dimensiones de la cuadrícula (rows y cols), y verifica que la casilla seleccionada corresponda a un pasillo transitable y no colisione con la estructura de un muro (`map[y][x] != '#'`).
3. El programa almacena los índices en las variables de instancia de la clase (`startI`, `startJ`) y posiciona el carácter identificador 'I' en la coordenada correspondiente.
4. De forma idéntica, el programa almacena los índices del punto de llegada en las variables de instancia (`endI`, `endJ`), posiciona el carácter identificador 'F' en la coordenada correspondiente y establece el booleano `entryExit` a `true`.

```
public void setStartEnd() {
    if (!loaded) {
        System.out.print("\nNo se ha cargado ningún laberinto");
        return;
    }
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (map[i][j] == 'I' || map[i][j] == 'F') {
                map[i][j] = ' ';
            }
        }
    }

    System.out.print("\nPara establecer la entrada indica las coordenadas\n");
    do {
        startI = Input.getXY("Valor de I: ");
        startJ = Input.getXY("Valor de J: ");
        if (startI >= rows || startJ >= cols || startI < 0 || startJ < 0) {
            System.out.print("\nValor no válido!!!\n");
        } else if (map[startI][startJ] == '#') {
            System.out.print("\nLas coordenadas coinciden con un muro\n");
        } else {
            map[startI][startJ] = 'I';
            break;
        }
    } while (true);

    System.out.print("\nPara establecer la salida indica las coordenadas\n");
    do {
        endI = Input.getXY("Valor de I: ");
        endJ = Input.getXY("Valor de J: ");
        if (endI >= rows || endJ >= cols || endI < 0 || endJ < 0 || (endI == startI && endJ == startJ)) {
            System.out.print("\nValor no válido!!!\n");
        } else if (map[endI][endJ] == '#') {
            System.out.print("\nLas coordenadas coinciden con un muro\n");
        } else {
            map[endI][endJ] = 'F';
            break;
        }
    } while (true);

    entryExit = true;
    System.out.print("\nEntrada y salida integrada correctamente!!!\n");
}
```

6.2.4 Seleccionar camino

Descripción Técnica: Gestiona la selección del algoritmo de resolución basándose en el tipo de búsqueda (ruta óptima o extendida) que elija el usuario.

Flujo de Implementación:

Control de errores: El método `Maze.seleccionarCamino()` verifica que las banderas `loaded` y `entryExit` sean verdaderas. Si alguna es falsa, detiene la ejecución e informa que falta cargar el mapa o fijar los puntos de inicio y fin.

Menú de selección: Superado el control, muestra las opciones por consola y captura la decisión con `Input.getInt()`. A partir de este valor, el programa muestra dos posibilidades:

- 1º Llamar a `Maze.shortPath()` pasándole la matriz y las coordenadas para ejecutar la búsqueda en anchura (BFS).
- 2º Llamar a `Maze.longPath()` con los mismos parámetros para iniciar la búsqueda en profundidad (DFS).
-

Finalización: Tras ejecutar el algoritmo establecido, el programa muestra el resultado en pantalla a través del método `Maze.showMaze` y muestra los pasos recorridos por el algoritmo con el método privado `Maze.printSteps`.

```
public void seleccionarCamino() {
    if (!loaded) {
        System.out.println("NO se ha cargado ningún mapa");
        return;
    }

    if (entryExit) {
        System.out.println("--- SELECCIONA UNA OPCIÓN ---");
        int opcion = Input
            .getInt("\n1º Encontrar cualquier camino\n2º Camino más corto\n0 Cancelar\n\nSelección: ");

        if (opcion == 1) {
            longPath(map, startI, startJ);
        } else if (opcion == 2) {
            shortPath(map, startI, startJ);
        }
    } else {
        System.out.print("Entrada y salida NO implementadas!!!");
    }
}
```

6.2.5 Ver usuario actual

Descripción Técnica: Proyecta la información personal y la ficha técnica del usuario autenticado en la sesión activa del sistema.

Flujo de Implementación:

1. Control de sesión: El método `Session.showUserInfo()` comprueba inicialmente que exista un usuario activo en el sistema (`user != null`).

2. Cálculo de la edad: A continuación, toma la fecha de nacimiento (en formato SQL) alojada en el objeto `User` y la envía a la función estática `Utils.getAge(dateSql)`. Este método utiliza la clase nativa `LocalDate` y la herramienta `Period.between()` para calcular de forma exacta los años cumplidos del usuario respecto a la fecha actual.

3. Impresión de datos: Una vez obtenido este valor entero, se invoca al método de instancia `User.userInfo(edad)`, el cual se encarga de formatear y mostrar ordenadamente en la terminal toda la ficha del perfil (ID, Nombre, Cuenta, DNI, Email, Domicilio y Rol).

```
public void showUserInfo() {  
    if (user != null) {  
        user.userInfo(Utils.getAge(Utils.formatDateSql(user.getBirthdate())));  
    }  
}
```

6.2.6 Cerrar Sesión (Logout)

Descripción Técnica: Invalida el estado de autenticación del usuario actual y restablece el entorno del programa al estado inicial (unlogged).

Flujo de Implementación:

Al ser capturada la opción en el menú principal por `Main`, se invoca el procedimiento `Session.logout()`. Este método modifica de forma inmediata el valor del atributo booleano de instancia `logged` pasándolo a `false`.

Para evitar vulnerabilidades de fuga de información en la memoria volátil del sistema, se desvincula la variable global estática de la clase `User` asignándole un valor nulo (`user = null`).

Por último, el método realiza un vaciado preventivo de las estructuras de datos del mapa invocando de manera interna a `Maze.clearMaze()`. Esto purga la matriz bidimensional de caracteres (`map = null`), forzando al flujo del programa a devolver real usuario a la pantalla del menú de inicio.

```
public void logout() {  
    logged = false;  
    user = null;  
    System.out.print("\n---Sesión cerrada---\n");  
}  
  
} else if (option == 6) {  
    session.logout();  
    maze.clearMaze();  
}
```

7º Desarrollo de la implementación del algoritmo del primer camino.

Este algoritmo tiene como objetivo encontrar un camino hacia la salida, sin garantizar que sea el óptimo o el más corto.

Utiliza una lista dinámica (ArrayList<Coordinate>) que se comporta de manera similar a una pila (LIFO - Last In, First Out), ya que el código extrae siempre el último elemento insertado (lista.remove(lista.size() - 1)). Esto provoca una búsqueda de resolución del laberinto de tipo Búsqueda en Profundidad (DFS), donde el algoritmo se desplaza de manera directa hacia el final de un camino antes de retroceder (hacer backtracking).

Para evitar que el programa entre en bucles infinitos, se utiliza una matriz bidimensional de booleanos (visitado[]) que marca las coordenadas ya exploradas, permitiendo al programa diferenciar las casillas no visitadas y las que ya han sido recorridas.

La exploración evalúa las celdas adyacentes en el orden definido por los vectores de movimiento: movementsY = { 1, -1, 0, 0 } y movementsX = { 0, 0, -1, 1 } (Abajo, Arriba, Izquierda, Derecha).

```
public void longPath(char[][] map, int startY, int startX) {
    limpiarMapa(map);
    Coordinate[][] aux = new Coordinate[rows][cols];
    boolean[][] visitado = new boolean[rows][cols];
    ArrayList<Coordinate> lista = new ArrayList<>();

    Coordinate start = new Coordinate(startX, startY);
    lista.add(start);
    visitado[startY][startX] = true;

    Coordinate end = null;
    int[] movementsX = { 0, 0, -1, 1 };
    int[] movementsY = { 1, -1, 0, 0 };

    while (!lista.isEmpty()) {
        Coordinate currentPos = lista.remove(lista.size() - 1);
        if (map[currentPos.y][currentPos.x] == 'F') {
            end = currentPos;
            break;
        }
        for (int i = 0; i < 4; i++) {
            int nextX = currentPos.x + movementsX[i];
            int nextY = currentPos.y + movementsY[i];

            if (nextY >= 0 && nextY < rows && nextX >= 0 && nextX < cols) {
                if (map[nextY][nextX] != '#' && !visitado[nextY][nextX]) {
                    visitado[nextY][nextX] = true;
                    aux[nextY][nextX] = currentPos;
                    lista.add(new Coordinate(nextX, nextY));
                }
            }
        }
    }

    if (end != null) {
        createPath(map, start, end, aux);
    }
}
```

8. Desarrollo de la implementación del algoritmo del camino más corto

El método shortPath está diseñado con un objetivo estricto: garantizar matemáticamente la ruta óptima (con el menor número de pasos posibles) entre la entrada ('I') y la salida ('F'). Para lograrlo, implementa un algoritmo de Búsqueda en Anchura (BFS - Breadth-First Search).

A diferencia de una búsqueda a ciegas, el funcionamiento de este algoritmo se puede entender perfectamente a través de las siguientes 4 fases:

1. Inicialización

El algoritmo comienza limpiando los restos de caminos previos en el mapa y preparando tres elementos clave:

- **La Cola (Queue):** Se utiliza una estructura FIFO (First In, First Out - Primero en entrar, primero en salir) implementada mediante una LinkedList.
- **El Origen:** Se introduce la coordenada inicial (start) en la cola y se marca como visitada en una matriz espejo de booleanos (`visitado[y][x] = true`) para no volver a pasar por ella.

2. Exploración Radial ("Por Olas")

El núcleo del algoritmo es un bucle que no se detiene mientras queden celdas por explorar en la cola:

Se extrae el primer elemento de la cola mediante `cola.poll()`, que se convierte en la `currentPos` (posición actual). Desde esa posición, el algoritmo mira de forma simultánea en las 4 direcciones posibles gracias a los vectores de desplazamiento:

```
movementsX = { 1, -1, 0, 0 }  
movementsY = { 0, 0, 1, -1 }
```

3. Registro de Rastro (`aux[][]`)

Al detectar una celda vecina válida (transitable y no visitada), el algoritmo realiza tres acciones automáticas:

- La marca como visitada (`visitado[y][x] = true`).
- Guarda de dónde viene: Almacena la coordenada actual en la matriz auxiliar (`aux[sigY][sigX] = currentPos`), lo cual es imprescindible para poder reconstruir la ruta hacia atrás al final.
- La añade a la cola para evaluar a sus propios vecinos más adelante.

4. Finalización del algoritmo

En el momento en que la celda extraída de la cola coincide con la salida (`map[currentPos.y][currentPos.x] == 'F'`), el algoritmo se detiene inmediatamente.

Debido a que la exploración se realiza de manera uniforme y por niveles de distancia (gracias a la estructura FIFO), la primera vez que se toca la salida se garantiza al 100% que se ha llegado por el camino más corto posible. No existe ninguna otra ruta con menos transiciones. Si la cola se vacía y nunca se encuentra la 'F', significa que el laberinto no tiene solución.

```
public void shortPath(char[][] map, int startY, int startX) {
    limpiarMapa(map);
    Coordinate[][] aux = new Coordinate[rows][cols];
    boolean[][] visitado = new boolean[rows][cols];
    Queue<Coordinate> cola = new LinkedList<>();

    Coordinate start = new Coordinate(startX, startY);
    cola.add(start);
    visitado[startY][startX] = true;

    Coordinate end = null;
    int[] movementsX = { 1, -1, 0, 0 };
    int[] movementsY = { 0, 0, 1, -1 };

    while (!cola.isEmpty()) {
        Coordinate currentPos = cola.poll();
        if (map[currentPos.y][currentPos.x] == 'F') {
            end = currentPos;
            break;
        }
        for (int i = 0; i < 4; i++) {
            int nextX = currentPos.x + movementsX[i];
            int nextY = currentPos.y + movementsY[i];

            if (nextY >= 0 && nextY < rows && nextX >= 0 && nextX < cols) {
                if (map[nextY][nextX] != '#' && !visitado[nextY][nextX]) {
                    visitado[nextY][nextX] = true;
                    aux[nextY][nextX] = currentPos;
                    cola.add(new Coordinate(nextX, nextY));
                }
            }
        }
    }

    if (end != null) {
        createPath(map, start, end, aux);
    } else {
        System.out.println("No se ha encontrado ningún camino");
    }
}
```

9º Conclusiones

La realización de este proyecto a lo largo de las distintas entregas ha supuesto un punto de inflexión en mi aprendizaje en este primer año de DAM. Lo que comenzó como una idea simple de pintar un mapa por consola ha evolucionado hasta convertirse en una aplicación completa, sólida y estructurada, permitiéndome asimilar de forma práctica conceptos teóricos que al principio de curso se veían complejos.

Uno de los aspectos más enriquecedores ha sido dotar al programa de mecanismos que simulan el software real. En el mundo real, un programa no puede "romperse" si el usuario se equivoca. Por ello, la integración de una robustez lógica mediante el control de excepciones, la validación estricta de coordenadas para evitar que la entrada o salida coincidan con un muro, y el filtrado en la lectura de ficheros externos me ha enseñado la importancia de programar de forma defensiva. Asegurar que el mapa esté correctamente cargado antes de permitir buscar un camino o forzar al usuario a introducir datos válidos mediante clases de apoyo (Input) dota al proyecto de una capa de seguridad esencial en cualquier desarrollo profesional.

Por último, el desarrollo de los algoritmos de resolución ha sido, sin duda, la parte más interesante del proyecto. Ha sido fascinante ver cómo un simple cambio en la estructura de datos que gestiona los movimientos transforma por completo el comportamiento del programa. El contraste entre ambos es muy evidente: mientras que longPath (con su lógica LIFO) explora el laberinto de una forma más "humana" y caótica, adentrándose por los pasillos hasta topar con pared, shortPath (con su lógica FIFO) se comporta de manera fría y eficiente, expandiéndose en forma de ola para asegurar de manera matemática la ruta óptima. Diseñar este sistema de rastreo mediante matrices auxiliares y trazar el camino inverso de forma limpia me ha dado una perspectiva real del potencial que tienen las estructuras de datos dinámicas en la resolución de problemas lógicos. En definitiva, este proyecto no solo ha consolidado mis bases en Java, sino que me ha motivado a seguir profundizando en la optimización de código y el diseño de software robusto.

10º Bibliografía

Oracle Java Documentation: <https://docs.oracle.com/en/java/>

Oracle JDBC API Guide: <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>

MySQL Connector/J Manual: <https://dev.mysql.com/doc/connector-j/en/>

GeeksforGeeks (Matrix Traversal): <https://www.geeksforgeeks.org/matrix-traversal-algorithms/>

W3Schools Java Tutorials: <https://www.w3schools.com/java/>